ACS-2947-002/003
Lab 7
Due by Wednesday, March 26, 11:59 PM*

Instructions

- The goal of this lab is to understand the implementation, inner working and use of maps. Please complete this lab before starting Assignment 4.
- Submit your `.java` files (together in a `Lab7.zip` file) via Nexus.
- Include your name and student number as a comment in every file.

***Note that the lab deadline is on Wednesday to allow us to finish the coding of the ChainHashMap in class. If you can, complete the lab earlier so that you can start Assignment 4.

Task

Write a program that populates simple data into a hashmap and deals with duplicate keys. Use a `ChainHashMap` for this exercise with buckets being instances of `UnsortedTableMap`.

Using the provided `Map` and `Entry` interfaces, provide the `ChainHashMap` implementation using the `AbstractMap`, `AbstractHashMap`, and `UnsortedTableMap` from your notes (L13_) or textbook as a base.

Create a class named `RGBColor` that stores a color `name` and three integers between 0 and 255 for each of the red, blue, and green values. Add other methods to this class and handle any possible exceptions as required.

In a `Lab7_Driver` class, create two hashmaps. The first map uses a color name as the `key` and `RGBColor` instance as the `value`. The second map uses a color name as the `key` and an integer as the `value`.

Populate the first hashmap with the provided RGB color data by reading data from the CSV file. When a duplicate key is encountered, determine how to address the key as follows:

a) If the duplicated keys have the same values i.e., the same name and the same set of RGB values, ignore the duplicated record and print a statement that it was ignored (*See sample output*).
b) If the duplicated keys have different values i.e., same color name but different RGB codes, store the name in the second hashmap together with the number of times the duplicate key has been encountered. Then add the duplicate record with a new color name – composed of the original name + an integer value between 1 and the number of duplicates of this color that has been seen. For example, the first time the color *Blue* is duplicated, the duplicate is added with key *Blue1*, a second duplicate is added with key *Blue2*, a third *Blue* duplicate is added with key *Blue3,* etc.
   - Read in the `data.csv` file
     - create each line as an instance of `RGBColor`
     - Use `nextLine()` of Scanner and `split(",")` of String to parse the CSV input

- Add each RGB color to the first hashmap
- When a duplicate key is encountered, note that your code will allow the previous value to be overwritten. Checking if two RGB codes are equal must be done using the `equals` method.
- Use the second hashmap to keep track of the number of each duplicate key as they are encountered.
- You do not need to create a `HashValue` method. Make sure that you understand how the `HashValue` method in our `AbstractHashMap` class works and uses the `hashcode` calculated by the `String` class.
- Since color names are strings, we can expect that our `HashValue` method in the `AbstractHashMap` class works correctly and uses the `hashCode` calculated by the `String` class.

After processing all the data, display the number of records added to the map.

Display the total number of colors that had a duplicate.

Display the number of duplicates that were encountered for each color with a duplicate.

Sample Output

```
Populating map from file:
Duplicate: Amaranth (62, 17, 41) replaced Amaranth (90, 17, 31)
Re-adding duplicate: Amaranth1 (90, 17, 31)
Duplicate: Amber (100, 49, 0) replaced Amber (100, 75, 0)
Re-adding duplicate: Amber1 (100, 75, 0)
Duplicate: Blue (12, 46, 100) replaced Blue (0, 0, 100)
Re-adding duplicate: Blue1 (0, 0, 100)
Duplicate: Blue (0, 58, 69) replaced Blue (12, 46, 100)
Re-adding duplicate: Blue2 (12, 46, 100)
Duplicate: Blue (0, 53, 74) replaced Blue (0, 58, 69)
Re-adding duplicate: Blue3 (0, 58, 69)
Duplicate: Blue (0, 9, 66) replaced Blue (0, 53, 74)
Re-adding duplicate: Blue4 (0, 53, 74)
Duplicate: Blue (20, 20, 60) replaced Blue (0, 9, 66)
Re-adding duplicate: Blue5 (0, 9, 66)
Duplicate: Blue (1, 28, 100) replaced Blue (20, 20, 60)
Re-adding duplicate: Blue6 (20, 20, 60)
...
Duplicate Ignored: Fuzzy Wuzzy (53, 26, 12)
Duplicate Ignored: Carolina blue (34, 63, 83)
Duplicate: Blue (0, 0, 100) replaced Blue (1, 28, 100)
Re-adding duplicate: Blue7 (1, 28, 100)
...
Duplicate: Green (0, 62, 42) replaced Green (0, 66, 47)
Re-adding duplicate: Green4 (0, 66, 47)
Duplicate: Green (0, 68, 26) replaced Green (0, 62, 42)
Re-adding duplicate: Green5 (0, 62, 42)
```

```
Duplicate: Green (0, 65, 31) replaced Green (0, 68, 26)
Re-adding duplicate: Green6 (0, 68, 26)
Duplicate: Green (40, 69, 20) replaced Green (0, 65, 31)
Re-adding duplicate: Green7 (0, 65, 31)
...
Duplicate: Mulberry (78, 31, 61) replaced Mulberry (77, 29, 55)
Re-adding duplicate: Mulberry1 (77, 29, 55)

Number of colors added: 575

Total number of duplicated colors: 28
Duplicate Copper colors: 3
Duplicate Gold colors: 3
Duplicate Lemon yellow colors: 1
Duplicate Gray colors: 1
Duplicate Cadet blue colors: 1
Duplicate Cerulean colors: 1
Duplicate Lavender colors: 1
Duplicate Green-blue colors: 1
Duplicate Blue-green colors: 1
Duplicate Chartreuse colors: 1
Duplicate International orange colors: 2
Duplicate Amaranth colors: 1
Duplicate Blue colors: 7
...
Duplicate Green-yellow colors: 1
```

Submission

Submit your **`Lab7.zip`** file that includes all the following files (`Lab7_Driver.java`, `Map.java`, `Entry.java`, `AbstractMap.java`, `AbstractHashMap.java`, `ChainHashMap.java`, `UnsortedTableMap`, `RGBColor.java`, any other file/classes you use) via **Nexus**.